Shorts Studio

The complete guide

You do not need to know how to code to use this. If you can copy and paste, you can run it.

Generated 2026-09-09

This guide covers everything: what the tool is, how to start it, how to make a video, how to keep your work safe with Git, and how to run the same tool on a second computer.

Contents

---

1. What this tool actually is

You pick a subject, press a button, and about three minutes later you have a finished video with a real human-sounding voiceover, ready to upload. Or you write the script yourself and let the tool do the rest — see Writing the script yourself.

Four services do the work, and they all run from one page in your browser:

	What it does	Needed?
Google Gemini	Writes the question, the four options, the explanation and the exact words the narrator says — or, in explainer mode, the whole storyboard. Also writes your title, tags and description at the end, and **draws the backdrop pictures** if you enable billing.	Required
Anthropic Claude	An alternative writer for the question or the storyboard. Pick which one on step 2.	Optional
ElevenLabs	Turns that script into speech. Can also draw backdrops, but only on a Pro plan.	Required
DeepSeek	Solves the question independently and says whether it agrees with Gemini.	Optional
Pexels + NASA	Free photos to sit behind the text. NASA needs no key.	Optional

The video itself is drawn on your own computer by **Remotion**, which is why rendering costs nothing.

Two ways to tell it

- **Quiz** — a question, four options, a countdown and the reveal. The format that stops a scroll.
- **Explainer** — no question at all. A storyboard that builds understanding scene by scene using analogies and diagrams: a title card, an analogy, a labelled diagram, the steps, a comparison, a timeline, a scene where things actually move, and a recap. Aimed at 3 to 5 minutes in 16:9.

Three kinds of video

- **Curiosity STEM** — counter-intuitive science and maths for a general audience.
- **Electrical exam prep** — in the style of a real paper, aimed at a specific exam: GATE EE, ESE/IES, SSC JE, RRB JE, State AE/JE, PSU (UPPCL/DMRC/NTPC/BHEL), or ITI/Wireman.
- **Aptitude & reasoning** — the quant, reasoning, English and awareness sections that nearly every competitive paper carries, aimed at SSC, banking, RRB, CAT, campus placements, GATE GA, CSAT, defence, state PSC or the teaching exams. See Aptitude and reasoning videos.

Two shapes

- **Portrait 9:16** — Shorts, Reels, TikTok. 30 to 90 seconds.
- **Landscape 16:9** — a proper explainer for YouTube. 2 to 5 minutes.

How long does one video take? Roughly 3–4 minutes of your attention, most of it waiting. The very first video takes longer, because the tool downloads its rendering engine once.

---

2. Setting up, once

Node.js

This is the program that runs everything. Check whether you already have it — press the **Windows key**, type `powershell`, press **Enter**, then type:

```
node -v
```

A reply like `v20.11.0` or higher means you are fine. An error means you need it: go to nodejs.org, download the big green **LTS** button, click Next through the installer, then **close and reopen PowerShell**.

Your API keys

An "API key" is a long password that lets this tool use an online service on your behalf.

Gemini (required — writes the questions)

1. Go to https://aistudio.google.com/app/apikey
2. Sign in with any Google account → **Create API key** → copy it.
3. It looks like `AIzaSyD…`

ElevenLabs (required — does the voice)

1. Go to https://elevenlabs.io and make a free account.
2. Go to https://elevenlabs.io/app/settings/api-keys → **Create API key** → copy it.
3. It looks like `sk_1a2b3c…`

Claude (optional — an alternative writer)

1. Go to https://console.anthropic.com → **API Keys** → **Create Key** → copy it.
2. It looks like `sk-ant-…`

Only needed if you want Claude to write the questions or storyboards instead of Gemini. See Using Claude instead of Gemini.

DeepSeek (optional — double-checks the answer)

1. Go to https://platform.deepseek.com/api_keys → **Create new API key** → copy it.
2. It looks like `sk-1a2b3c…`

Worth adding. Gemini writes the question *and* marks its own answer, so nothing catches a confidently wrong one. A check costs a fraction of a cent.

Pexels (optional — backdrop photos)

1. Go to https://www.pexels.com/api/ → sign up free → **Get Started** → copy the key.

Without it you still get NASA's public-domain library, which is excellent for space and physics but thin for chemistry and biology.

> **Keep these private.** Anybody holding your key can spend your credits. Never put one in a screenshot, a video, or a message. They are stored only in your browser, never in a file — which is also why they are never uploaded to GitHub.

Install the packages

Open PowerShell and go to the project:

```
cd C:\Projects\shorts-studio
```

Then:

```
npm install
```

This takes one to three minutes. Yellow warnings are normal — only a red `ERR!` means trouble.

If you see "running scripts is disabled on this system"

> This is a Windows security default, not a fault in the tool. Two ways past it.
>
> **The easy way, changing nothing** — add `.cmd`:
>
> `npm.cmd install`
> Use `npm.cmd` everywhere this guide says `npm`. That is the whole fix.
>
> **The permanent way** — run this once and press `Y`:
>
> `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
> `RemoteSigned` lets scripts written on your own computer run, while anything downloaded from the internet must still be signed. It applies to your account only, and it is what Microsoft recommends for people who develop on Windows.

---

3. Starting it, every time

```
cd C:\Projects\shorts-studio
```

```
npm start
```

Two things start together: the **helper**, which talks to Gemini and ElevenLabs and does the rendering, and the **web page** you actually use. The helper opens your browser once it is ready, so give it a few seconds. If it does not appear, paste this in yourself:

```
http://localhost:5173/
```

That address never changes. If something else is already using it the tool stops and says so, rather than quietly moving to another one.

> **"Starting up..." for a second or two is normal.** The page loads faster than the helper does, so on a cold start it waits for it and tells you. It clears by itself. Only if it is still trying after twenty seconds will it tell you the helper is genuinely missing.

Leave the PowerShell window open the whole time. That black window *is* the engine. Closing it stops the tool. When you are finished for the day, click it and press `Ctrl + C`.

---

4. Making a video — the seven steps

Seven numbered buttons run across the top. You move left to right. Steps you have not earned yet stay greyed out on purpose, so you cannot get lost.

Step 1 — Keys

Paste your keys and press **Test my keys**. You want green ticks. It also reports how many ElevenLabs voice credits you have left this month.

You do this once, ever. If a key is rejected, the cause is almost always a stray space — clear the box and paste again.

Step 2 — Topic

Where you decide what the video is about.

Setting	What it does
How it is told	**Quiz** or **Explainer**. Choosing Explainer switches to 16:9 and at least three minutes, because that is what the format needs.
What kind of video	**Curiosity STEM** for a general audience, **Electrical exam prep** for candidates revising, or **Aptitude & reasoning** for the quant, reasoning, English and awareness sections.
Exam *(both exam modes)*	Sets the depth. GATE and ESE want derivation and analysis; SSC and RRB want standard formulas at speed; ITI wants practical wiring and safety with no calculus. In aptitude mode the list swaps to the aptitude papers, where CAT hides an insight and SSC stays quick and clean.
Format	Portrait or landscape. **Pick this first** — it changes how much script is written and how the frame is laid out.
Subject	The broad area. In exam mode this is the syllabus section.
Sub-topic	A dropdown of suggestions for that subject — 38 for Power Generation, for example. Pick one to concentrate a run of videos on a single section, or leave it on *Any*.
Or type your own topic	Free text. Whatever is in this box is what gets used; clear it to let Gemini choose.
Who is watching	Sets vocabulary and assumed background.
Difficulty	*Very easy* through *Brutal*.
Question style	*Mathematical* = the viewer must calculate. *Theoretical* = they must reason. *Real-world* = anchored in daily life.
Narration language	The language spoken *and* shown. Kannada, Hindi, Tamil and Telugu render in their own script — pair with a matching voice in step 4 and the Multilingual v2 model.
Curiosity factor	The most important dial. **8 or 9** is the sweet spot: counter-intuitive enough to make people comment.
Target length	40–50s performs best for Shorts. 180–240s is the sweet spot for explainers.
How many diagrams	*Rich* (the default) puts a chart, circuit or animation on nearly every scene. *Balanced* on most explanation scenes. *Sparse* only where it really helps.
Gemini model	Flash is fast and cheap. Pro is worth it for hard maths. The list is built from your own key, so it only offers models you can actually use.

Curiosity high — what is trending now

Press **What is trending now?** and Gemini *searches the live web* — it is not answering from memory — then judges which of what it finds would actually make a good video. You get eight ideas, each with why it is being

talked about and the counter-intuitive angle to build around. Click one and it fills the topic box.

It searches your domain: science and technology in curiosity mode, grid, generation, storage and standards news in electrical mode, or exam calendars, paper-pattern changes and the current affairs a general awareness section actually asks about in aptitude mode. **Where this came from** lists the pages it actually read, so you can check a claim yourself.

> **Trending is not the same as true.** A story spreading fast is exactly the kind that turns out to be half right. Verify the answer in step 3, and run the DeepSeek check if you have a key.

This needs a **2.5 model** (Flash or Pro) — older models cannot search. It costs one Gemini request.

Your intro (optional). Type a greeting — "*Hi, it's Hemanth here. Ready for today's question?*" — and it becomes the first thing the video says. There are one-click presets. **Gemini never rewrites this**: whatever you type is spoken and shown word for word.

Press **Generate the question**. Five to twenty seconds.

Step 3 — Script

You check Gemini's homework. **This is the most important step and the one people skip.**

> **Only the spoken words appear on screen.** The narration *is* the on-screen text — shown a few words at a time, each word lighting up as it is said. There is no separate headline to keep in step, and no labels, step counters or subject chips cluttering the frame.

Every line is editable. **Read the question and satisfy yourself the marked answer is right** — AI is confidently wrong sometimes, and ten seconds here saves publishing something embarrassing.

The second opinion. With a DeepSeek key you get a **Check the answer** button:

Badge	Meaning
✓ **DeepSeek agrees**	Both models got the same answer.
⚠ **Reservations**	The answer stands, but something needs a look.
✕ **DeepSeek disagrees**	They got **different answers**. One is wrong.

On a disagreement you get a one-click button to switch the marked answer — but read the reasoning first. DeepSeek is not automatically right either; the value is knowing to look. Pick **Reasoner** for hard maths.

Edit the question afterwards and the badge is replaced by "*the check is out of date*" — a stale tick is never left pretending the new version was verified.

Watch the length. A bar shows how many seconds you have written against the length you asked for. If a generated script came out well under target you also get a warning — regenerate now, while it is still free.

> The bar is an estimate at the voice's own speaking pace. **You cannot set how long a scene lasts**, and there is no slider for it anywhere. A scene lasts exactly as long as its recorded narration — that is the whole reason the words on screen stay locked to the voice. The real length arrives with the voiceover in step 4.

Rearranging the video. Every scene has controls in its header:

Control	What it does
The **kind** dropdown	What the scene is for — hook, question, options, countdown, answer, explanation, outro.
↑ ↓	Move the scene earlier or later.
×	Delete it.
+ Add a scene	Adds an explanation scene before the outro.

Changing any of these means the voiceover has to be recorded again, and the page tells you so if you have already made one.

On an explainer, a scene that draws a layout shows a 🖼 **Drawn on screen** row listing every label in it — and on a motion scene, every cue that fires an animation. A label marked ⚠ is one your narration never says — the reveals are timed by matching the spoken words against those labels, so an unmentioned label can only appear on a guess. Mention it in that scene's narration, in the order shown, and the warning clears.

Step 4 — Voice

Pick a voice from your own ElevenLabs account and press **Hear this voice** to sample it. If the list is empty, add any voice from the Voice Library and reload.

The cost is shown before you spend anything. Press **Make the voiceover**.

> **This is where sync happens.** Every clip is decoded in your browser and measured for real — exact length, where sound starts, where it stops. Each scene is then made exactly as long as its own clip, always rounding *up*, so a line can never be cut off. The subtitles are nudged by the few milliseconds of silence every MP3 carries at its start, which lands the highlight on the right syllable.

The options light up as they are read, because the narration reads them in order.

Open **Show the sync report** to see the numbers per scene. If a line ever sounds clipped, turn off **Trim trailing silence** in step 5.

Step 5 — Look

Everything here is instant, free, and never touches the voiceover.

- **Dark or light**, and four layouts: **Simple** (clean), **Elegant** (serif, documentary), **Nerdy** (terminal green on graph paper), **Flashy** (loud, best in a feed).
- **Highlight colour**, **thinking time** (3–5s), **breathing room**.
- **Show the spoken words** — the read-along text. Leave it on; most people watch on mute.
- **Draw the diagrams** — on the explanation and outro scenes, and a **setup diagram on the question scene**: the circuit, the apparatus or the geometry being asked about. Anything that could hint at the answer is stripped from that one automatically — no charts, no pies, no comparison panels, and no caption. As well as static ones (a formula box, comparison bars, a side-by-side panel, an icon) Gemini can choose a **live animation** from a fixed library of ten: wave interference, a travelling wave, orbits, a projectile arc, a pendulum, a vector field, spreading particles, a graph being drawn, an atom, light refracting — plus six built for electrical and power work: a **circuit** (series or parallel), a **phasor diagram** and power triangle, an AC **waveform** (phase shift, rectified, PWM), a **block flow** that lights up stage by stage (boiler → turbine → condenser → pump), a **transformer** with turns ratio, and a **pie** for a fuel mix or a loss breakdown. There are now **over two hundred**

of these — enough that every sub-topic the tool offers has one that fits — covering mechanics (levers, pulleys, gears, springs, collisions, friction, torque), light and sound (reflection, lenses, prisms, the Doppler effect), heat and fluids, chemistry (molecules, pH, titration, electrolysis, reaction profiles), biology (cells, DNA, neurons, the heart, photosynthesis, food chains), maths and geometry, data charts, earth and space, and sixteen built for aptitude and reasoning — a **Venn** diagram, a **clock face** with the angle between the hands, a **number line**, a **ratio bar**, a **seating arrangement**, a **family tree**, a **histogram**, a **logic grid**, **cube nets**, **dice**, **paper folding**, **mirror images** and more. Gemini is told not to use the same one twice in a row, and the tool drops it if it does anyway.

- **Cuts and text** — how one scene becomes the next, and how the words arrive. See Cuts, text and the finishing layer.
- **Finishing layer** — grain, a vignette, cinema bars. Same section.
- **Drift topic symbols** — faint themed emoji behind everything.
- **Moving backdrop** — one of thirty slow animations under the whole video. See Moving backdrops below.
- **Backdrop photos** — press **Find backdrop photos** and it searches Pexels and NASA per scene, using a search term Gemini wrote for that scene. **Nothing is applied for you**: a photo library will cheerfully return a beach for "gravity". Click the ones that fit, skip the rest.
- **Drawn backdrops** — if your ElevenLabs plan allows it, each scene also gets a **Draw** button that makes a picture instead of finding one. See Drawing your own backdrops.
- **Sound** — three built-in music beds (calm, tense, upbeat), or load your own file. The music **ducks automatically** under the narration. Effects: a countdown tick, an option whoosh, an answer chime, and a sweep between scenes.

Play the phone preview before rendering. Fixing something here takes a second; after a render it takes minutes.

Step 6 — Export

Choose a quality (**Normal** is right almost always) and press **Render the video**.

> **The first render is the slow one.** The very first time, the tool downloads a rendering browser of about 150 MB. It can look like nothing is happening for several minutes. It only ever happens once per computer.

A 45-second video takes one to three minutes. You get a player, a download button, and the file is saved into the `out` folder automatically.

Step 7 — Publish

Gemini writes the metadata from the finished video: **several title options** to choose from, a description, tags, hashtags, suggested thumbnail text, and a pinned comment. Each has a copy button, and all of it goes into the upload kit at the bottom of the step.

For exam-prep videos the title and first line of the description lead with the exam name, subject and topic, because that is what people actually type into search.

Making a thumbnail

Step 7 has a **Thumbnail** section. It renders an image in the same colours as the video, on your own machine, so it costs nothing and you can make as many as you like.

Pick the shape first. It defaults to the shape of the video you just made.

Shape	Size	For
16:9	1280 × 720	The YouTube cover image on a normal video.
9:16	1080 × 1920	Shorts, Reels and TikTok.

The layouts adapt rather than being cropped: in 9:16 the **split** symbol moves above the text and the **question** mark sits on its own line, because a narrow frame cut into two columns leaves both too thin to carry anything. Everything is kept to the middle of a portrait frame on purpose — the apps put their own title, channel name and buttons over the top and bottom.

Layout	Use it when
Statement	One bold claim. Works for most videos.
Question	A huge **?** beside the text. Classic quiz thumbnail.
Number	The answer is a figure — 8,760 · 60% · 50 Hz. The strongest of the four when it fits.
Split	Text and one big emoji — side by side in 16:9, stacked in 9:16.

Wrap a word in `*asterisks*` to colour it with your accent — `hits the ground *first*` puts *first* in the highlight colour. It is the single thing that makes a thumbnail read as designed rather than typed.

If you wrote the title and tags first, the **thumbnail text** Gemini suggested is filled in for you.

> **Judge it in the small box.** Two previews appear: one at the width the thing is really seen at — about **320** pixels for a 16:9 row, about **200** for a portrait shelf — and one full size. The small one is the honest test. If you cannot read it at a glance there, cut words out — six or fewer is the target, and the field warns you above that. Shrinking the type to fit more in is what makes thumbnails invisible.

The PNG is saved into `out\` next to your videos. Press **Save the PNG** to put it wherever you like.

The upload kit

The boxes on step 7 are gone the moment you close the tab, and the upload usually happens later — on another day, or from another machine. **Pack the upload kit** puts all of it in one zip beside your video, along with the thumbnail.

```
how-does-a-fish-get-past-a-dam-upload-kit.zip
├── UPLOAD.txt           the whole form, in order, with the character counts checked
├── title.txt            one field per file, for fast copy-paste
├── description.txt      complete and paste-ready — chapters and credits already in it
├── tags.txt
├── hashtags.txt
├── pinned-comment.txt
├── chapters.txt
├── credits.txt
├── metadata.json        if you ever script the upload
└── thumbnail.png
```

Open **UPLOAD.txt** first. It walks the YouTube form field by field, counts every character against the real limit, and shouts if the title is over 100 — silently trimming it would hand you something YouTube cuts off mid-word, which

you would only notice after publishing.

Two things in the kit cannot be copied off the page, because only the tool knows them.

Chapters are real timestamps, computed from the scene timings that produced the video. Nobody can type these accurately afterwards. They are already inside `description.txt`, so pasting that one block gets you the chapter bar under the scrubber for free.

You do not always get them, and that is deliberate. YouTube ignores a chapter list unless it starts at `0:00`, has **three or more** marks, and none is **under ten seconds** — and it does not tell you it has ignored it. So short scenes are merged, and a video that cannot have a valid list gets none rather than a broken one. Shorts never get chapters.

Credits lists what actually ended up in this video and what each thing asks for in return:

What is in the video	What it asks for
Icons	Named per set. Anything wanting a credit is flagged, and the exact line to paste is written for you — it is already in the description.
Photos	Any stock you picked, with its credit line.
Music	Always yours. It is synthesised here from scratch — not sampled, not licensed from anyone, so no copyright claim is possible.
Narration	Flagged as a synthetic voice, since some platforms want that disclosed.

Pack it again any time — after changing the title, or after making a thumbnail. It rebuilds from whatever is on the page at that moment.

Using Claude instead of Gemini

Add a **Claude API key** on step 1 and a **Who writes it** choice appears on step 2. Pick **Claude** and the question or the storyboard is written by Claude instead; pick **Gemini** and nothing changes.

The choice only appears once a Claude key is present — a button that could only fail is worse than no button. Remove the key later and the tool quietly goes back to Gemini; your preference is remembered, so pasting the key back returns you to Claude.

The **Claude model** dropdown replaces the Gemini one and is built from your own key, newest and most capable first — Opus above Sonnet above Haiku. Opus is the default.

The rest of the tool does not care which one wrote it. The review gate, the voice, the diagrams, the render — all identical.

> **Two steps still use Gemini, whichever you pick. What is trending now** needs live web search, and the title and tags on step 7 are written by Gemini. Keep your Gemini key even if you write with Claude; step 2 reminds you.

On cost — read this before switching. Gemini has a generous free tier. Claude has **none**: you are billed from the first request, and you have to add a payment method before any key will work.

Rough cost of one video's worth of writing, at the published rates:

	Claude Opus 5	Claude Sonnet 5
One quiz	~9c	~4c
One five-minute storyboard	~35c	~14c

Estimates, not quotes — the real figure moves with how long the script is and how much the model thinks. Two things follow from them. **Output dominates**, so the storyboard costs far more than the quiz and the model choice matters more than anything else you can change. And **a day of making shorts on Opus is a few dollars, not a few cents** — set a spend limit in the Anthropic console so a runaway loop cannot surprise you.

If cost matters more than the last few percent of quality, Sonnet is the sensible default here.

Writing the script yourself

You do not have to let Gemini write it. On step 2, set the **format** and the **target length**, then press ✍ **Write it myself** instead of Generate.

That drops you straight into step 3 with an empty script of the right shape — the correct beats in the correct order, and about the right number of them for the length you chose. A 180-second explainer gives you 18 scenes to fill in; a 45-second quiz gives you the question, the four options, the countdown, the answer and a few explanation cards.

It costs nothing and needs no Gemini key. Credits are only ever spent on step 4, the voice.

In step 3 every scene has a **kind** dropdown, ↑ ↓ to reorder and ✕ to delete, and there is an **Add a scene** button underneath. A bar shows how many seconds you have written against the length you asked for.

> **One thing that is not a slider.** You cannot set how long a scene lasts. A scene lasts exactly as long as its recorded narration — that is the whole reason the words on screen stay locked to the voice. The bar in step 3 is an estimate at the voice's own speaking pace; the real length arrives with the voiceover in step 4.

Real artwork in every layout

A diagram box used to hold a label and, at best, one emoji. Now every box, every process step and every grid cell can carry a **drawing of the thing it names** — pulled from the same open library of about 200,000 icons the moving scenes use.

The storyboard writes a plain English noun — `boiler`, `turbine`, `fish`, `battery` — and the tool finds it. The drawing arrives in **your theme's colours**, so it never looks pasted in from somewhere else, and it brightens as the narration reaches it.

This is why a layout stops looking like a row of empty rectangles, and it costs nothing: the drawings are fetched once when the script is written and travel inside it from then on.

The storyboard is told to name a picture wherever a **real object** is on screen, and to leave it out for an abstract idea — there is no useful drawing of *efficiency*, and a wrong picture is worse than none. Where nothing matches, the layout falls back to the emoji or to plain text, so a missing icon costs you one shape rather than the scene.

Nothing is ever completely still. Things that hold themselves up in a fluid — a fish, a bird, a balloon — keep swimming on the spot. Flames flicker. Anything that turns keeps turning. It is small enough that you will not consciously see it and large enough that its absence is what makes a frozen sticker look like a frozen sticker.

Animation comes from your verbs

Every explainer scene used to animate once as it appeared and then hold perfectly still. On a twenty-second scene that is about one second of movement and nineteen of a screenshot — which is what makes a video feel flat even when every frame is correct.

Now the narration drives it. **The words you write are the animation.** When the voice says *flows*, something flows, at that moment. Say *spins* and something turns. Say *escapes* and it bursts outward. Nothing is authored, nothing is scheduled — it is read off the script.

Say	You get
flows, pours, travels, carries, circulates	particles crossing the frame
rises, climbs, grows, increases, expands	an upward drift
falls, drops, sinks, decreases, collapses	a downward drift
spins, rotates, revolves, orbits, turbine	soft arcs turning behind it
heats, burns, boils, combustion	a warm wash and rising haze
cools, freezes, condenses	a cold wash
collides, blocks, strikes, barrier, bounces	a shove and a flash
vibrates, oscillates, alternates, resonates	standing waves
voltage, current, charge, circuit, spark	electrical flecks
escapes, releases, bursts, erupts	particles thrown outward
glows, shines, brightens, radiates	a bloom
rain, droplets, liquid, leaks, floods	falling droplets

Step 3 shows an ✨ **Moves on these words** row under each scene, so you can see what a scene will do before you record anything. That row is also the control: change "*the steam goes into the turbine*" to "*the steam **flows** into the turbine*" and the scene gains a flow. **Write the verb you mean.**

Restraint is built in. At most four per scene, never two within about two seconds, always behind the text, and always low contrast. An effect is meant to be felt, not watched — the moment your eye goes to the particles instead of the diagram, it has failed. Words that are common English but rarely a real movement (*light*, *up*, *down*) are deliberately left out, because an effect firing on a sentence that did not mean it is worse than one that never fires.

Effects are aimed, not just timed. When the voice says "*the turbine spins*", the spin appears **on the turbine box** — not in the middle of the frame. The tool works out which item the narration has reached, from the same reveal order the panel itself uses, and fires the effect there. A flow passes through the row the active box is on; a bloom, a burst and a heat wash all centre on it.

That is most of the difference between an effect that explains something and one that is just decoration laid over the top.

Scenes now cross into each other. Each scene is held on screen a third of a second past its narration, so the outgoing and incoming ones overlap: one fades and slides out as the next fades and slides in, moving the same direction through the join. It used to be a hard cut with a five-frame flicker in front of it.

Three other things now move on their own, whatever you write:

- **Arrows carry something.** A diagram arrow used to draw itself once and freeze. Now a pulse travels down it for as long as the diagram is up — an arrow means *this goes to that*, and a still line is the one thing that cannot show it.
- **Process steps are joined by live connectors** rather than a → that only changes colour.
- **Every scene pushes in slowly** — about four percent over its length. Nobody consciously notices it; everybody feels the difference between a layout that was filmed and one that was screenshotted.
- **Things that should never be still, are not.** A fish holds station by swimming, a flame flickers, anything that turns keeps turning.

Scenes that move

Most explainer layouts show a **structure** and light parts of it up as the voice reaches them. One does something different: **motion** acts an event out.

A salmon swims at a dam. It is thrown back, twice. A fish ladder appears beside the dam, and the salmon climbs over it one step at a time. That is one scene, and the storyboard writes it in about eight lines.

Where the pictures come from. The storyboard names things in **plain English** — "fish", "dam", "turbine" — and the tool finds each one in an open icon library of about 200,000 shapes. They are fetched once when the script is written, cached, and then live **inside your script**. Nothing is downloaded while the video renders, so a render is as fast and as repeatable as any other, and works offline once the script exists.

Icons are drawn in your theme's own colours, so they never look pasted in from somewhere else.

Seven things can happen. The storyboard picks from a fixed list — it cannot invent an eighth:

Verb	What you see
appear	Fades and pops in. For something that arrives partway through.
move	Travels across and stops *beside* another thing.
blocked	Runs at something, is thrown back, tries again, gives up.
climb	Steps up and over something — the way through, once one exists.
pulse	Swells once, to say *this one, now*.
spin	Rotates on the spot.
exit	Drifts away and fades.

What times it. Nothing here runs on a stopwatch. Each beat carries a **cue** — a word or two from that scene's narration — and fires when the voice reaches it. The salmon is thrown back on the words *"wall of concrete"*, not at 4.2 seconds. Rewrite the narration and the animation follows it.

That is also why a motion scene shows the same ⚠ warning as any other layout in step 3: a cue your narration no longer contains is a beat that can never fire. Put the words back and it clears.

When you get one. The storyboard is told to include one motion scene wherever the subject has a moment that genuinely moves — something blocked, carried, escaping, or finding a way past an obstacle — and to put it in the middle, where the mechanism is being explained. Two at most. On a subject that does not move, none is the right answer and you will get none.

To ask for one directly, put it in **Extra instructions** on step 2:

> *act out the mechanism with a moving scene*

The tool checks the storyboard and tells you. A motion scene fails quietly — a beat whose cue is missing still animates, just on a guess instead of on the voice — so the black PowerShell window says so the moment the script arrives, while regenerating is still free:

```
[generate] check: scene 4: the narration never says "sheer bulk", so those beats cannot fire on the voice.
[generate] check: scene 4: "collapses" is cued on the last few words, so that beat will barely be seen.
[generate] check: scene 4: "fish" and "dam" start almost on top of each other.
```

None of these break the video. They tell you it will be looser than it should be. Regenerating usually clears them; so does rewriting that scene's narration to contain the missing words.

One bad cue does not spoil the rest. Each beat is matched to the voice on its own, so if the model writes three good cues and one it never says, the three still land exactly where they should and only the fourth is guessed at.

Things travel at a speed, not for a time. A moving actor used to take the same 1.15 seconds whether it crossed a tenth of the frame or three quarters of it, so long journeys were flung and short ones stalled. Distance now sets the duration. Verbs that cover more ground than the straight line — a bounce goes in and back twice, a climb zig-zags up in steps — get proportionally longer.

Two beats never run at once. When the narration puts two cues close together, the first beat stops where it got to and the second carries on from there. They used to both keep going and add their movements together, which is the lurch that looked like the animation glitching.

Nothing is drawn over the words. Actors are kept inside a band that clears the scene title above and the caption band below, whatever coordinates the storyboard writes — including a beat that names an explicit destination.

If a video still feels busier than it should, step 5 has a **Narration effects** → **How strong** slider. Turn it down to calm everything the script triggers; at 0 the layouts hold still. It does not touch the moving scenes themselves — those are content, not decoration, and are turned off with **Draw the diagrams**.

> **Ask for it by asking for an event.** Motion is for something *happening* — being blocked, finding a way through, escaping, being carried. For a list, a comparison or a structure, the other layouts are better, and the storyboard is told to use them instead. Expect one or two motion scenes in a video, not six.

A note on credits. The tool prefers icon sets that ask for nothing in return — MIT and Apache licensed. If it ever has to fall back on a set that wants a credit, it says so in the black window when the script is written, and names the set to put in your description.

Moving backdrops

Step 5 has a **Moving backdrop** — a slow animation under everything, so a scene reads as produced rather than as text on a flat colour. Thirty of them, in ten families: drifting particles, flow fields, constellations, waves, orbits, a circuit-board grid, light beams, spirals, equaliser bars and contour maps.

Auto picks one from your subject: a circuit board for electronics, a lattice for transmission, equaliser bars for power generation, a galaxy for astronomy, contours for climate.

It is capped well below the text at every setting, so it can never make a caption hard to read. Turning it up makes it busier, not more distracting. It costs roughly **20% more render time** with the heaviest of them; set it to *None* if

you want the fastest possible render.

Drawing your own backdrops

The ElevenLabs key that speaks your script can also **draw** the picture behind it. On step 5 every scene gets a **Draw** button next to its search box, beside the free Pexels and NASA search.

Why bother, when photos are free? Two reasons.

The first is shape. Stock libraries are full of wide photos taken for wide screens. Your short is tall, so the app has to cut a tall slice out of the middle of a wide picture — and whatever was happening at the left and right edges is simply gone. A drawn picture is made 9:16 (or 16:9 if that is what you chose) from the start, so nothing is thrown away.

The second is consistency. Twelve stock photos are twelve photographers, twelve lighting setups and twelve colour grades. Pick a **Look** — editorial photo, clean diagram, cinematic, or blueprint — and every scene you draw in that video comes back matching the others.

How to use it

1. Check the words in each scene's box. That is what gets drawn, so make it specific.
2. Pick a **Look** and an **Image model**. Flash models answer in a few seconds and cost the least; Pro is slower and sharper.
3. Press **Draw** on a scene. The picture appears among that scene's options with a purple **AI** tag and a dashed border.
4. Click it to actually use it — same as a photo. Not clicking costs you nothing further.
5. Not right? Press **Draw another**. The first one stays, so you can compare them side by side.

Which service draws it. There are two, and you pick with the tiles above the Draw buttons.

- **Google** *(recommended)* — uses your **Gemini** key. Needs **billing enabled** on that key's project; there is no free tier for image generation on any Google model. About **3p an image**, so roughly 25p for a whole video. Writing the script stays free either way.
- **ElevenLabs** — uses your ElevenLabs key, and needs a **Pro plan**. The free and Starter tiers cannot draw at all.

Write what to draw. Each scene has a **prompt** link next to its Draw button. The search words above it were written for a *photo library* — two or three nouns — and an image model given two nouns draws two nouns. The prompt box is where you describe the picture instead.

- **draft one from the narration** turns what the scene actually says into a starting point. It drops the words that only work out loud — "*so*", "*you*", "*we*" — because an image cannot show the listener. On a hook or an outro it uses your topic instead: "*Most people get this wrong*" describes the viewer, not a picture.
- **Last sent** shows the exact prompt that went to the model, so a redraw is a correction rather than another guess.
- Leave it empty and the search words are used, exactly as before.

Whatever you write, the **Look** and the composition rule are still added on the end — so an edited prompt keeps the style match and keeps the middle of the frame clear for your captions.

Keep every scene in the same style. On Google only, and on by default. The first image you draw becomes the reference for every one after it, so a video looks like one set instead of twelve unrelated pictures. It costs nothing extra.

What it costs. On ElevenLabs, every press spends credits from the same balance as your voiceover, and drawing through the API needs a **Pro plan or above**. On the free or Starter plan the voice still works perfectly — only the Draw button will tell you the plan is not enough. There is deliberately no "draw every scene" button: it would be one click to spend a lot of credit on scenes you were going to skip anyway.

Being honest about it. Drawn backdrops are labelled *Generated with ElevenLabs* in your caption and in the publish kit's credits file. Leave that in. YouTube and the other platforms increasingly expect AI-made imagery to be declared, and the line is short enough that it costs you nothing.

Cuts, text and the finishing layer

Three settings on step 5 that apply to every scene. Because they run on all of them, a choice here is *felt* across a whole video rather than noticed once — which is exactly why the defaults are the quiet options.

The transition — how one scene becomes the next

Ten choices. **Auto** is the default and is usually the right answer: it varies the join by what the scene is doing, so the answer reveal gets the punchy zoom, the question gets a wipe, and the explanations get the quietest crossfade there is. A single transition used forty times becomes a tic by the fourth scene.

	What it does	When
Auto	Varies with the scene	Leave it here unless you want one look throughout
Crossfade	Dissolve with a small drift	Never wrong, never noticed
Slide	The new scene moves in over the old	Clean and modern; reads well on a phone
Push	The old scene is shoved out by the new	More physical; good for step-by-step
Wipe	A hard edge sweeps across	Graphic and confident; suits bold layouts
Zoom	Punches in through the cut	Energetic; best on short, fast videos
Blur	Defocus and refocus	Soft and expensive-looking; slows the pace
Dip	Through the background colour	A real beat; use when two ideas are separate
Whip pan	Fast sideways smear	High energy — not for twenty scenes
Glitch	Digital break-up	Loud. One or two a video, not every cut

How the words appear

The read-along model never changes: one line on screen, exactly what you are hearing, the spoken word lit. What you are choosing is the *manner of arrival* — and the real decision inside it is whether the viewer may read slightly ahead of the voice.

- **Fade** *(default)* — the phrase appears and words light up as they are said. Unspoken words wait faintly, so the eye can run a shade ahead. Calmest, and the easiest to follow.
- **Pop** — each word snaps in as it is spoken. Punchy; suits hooks.
- **Rise** — words lift into place. Smooth all-rounder.
- **Typewriter** — nothing exists until it is said. Nothing to read ahead to, so it holds attention harder. Use it when you want urgency, not comprehension.

- **Focus** — unspoken words are blurred and sharpen as they are reached.
- **Highlighter** — a bar sweeps behind the current word. The strongest choice for **method and revision videos**: it reads as teaching rather than as motion graphics.
- **Bounce** — springy overshoot per word. Best with the Flashy layout.

The finishing layer

A texture over the top of everything. Unlike the moving backdrop, it means nothing and is not chosen from your subject — it is there to make a frame look *shot* rather than assembled.

Vignette (darkened edges), **film grain**, **scanlines** (CRT), **light leak** (a drifting warm bloom), **cinematic bars** (letterbox), **corner frame** (viewfinder brackets), **dust** (floating specks), **chromatic edges** (a lens fringe). Plus **None**, which is the default and what every video looked like before this existed.

Each has its own ceiling, so **How strong** at maximum cannot make the captions hard to read — a vignette can afford far more than grain can. If you are unsure, vignette at half strength flatters almost everything and is impossible to notice.

> One at a time. These stack with the moving backdrop, the drift symbols and the backdrop photo, and a frame carrying all four is a frame with nothing to look at.

Aptitude and reasoning videos

Almost every competitive exam in India carries an aptitude paper alongside the technical one, and for a lot of candidates that is the paper which actually decides the result — everyone revises their own subject, and nobody practises ratios. **Aptitude & reasoning** on step 2 writes for those sections.

What it covers. Twenty-eight sections, in the three blocks a paper is printed in:

- **Numerical** — number system, simplification, percentage and ratio, averages and alligation, profit and loss, interest, time-speed-distance, time and work, algebra, geometry and mensuration, trigonometry, permutation and probability, data interpretation, data sufficiency.
- **Reasoning** — series, coding-decoding, blood relations and directions, syllogism, puzzles and seating arrangement, analogy and classification, non-verbal, analytical and critical reasoning, clocks-calendars-cubes.
- **The rest of the paper** — English and comprehension, current affairs, static GK, general science, computer awareness.

Each one comes with its own sub-topic list, so you can run a whole series on boats and streams, or on syllogism possibility cases, without typing a topic each time.

Pick the paper, not just the section. The **Exam** dropdown matters more here than it looks. An SSC quant question and a CAT quant question can sit in the same chapter and be nothing alike: SSC wants one clean step in about forty-five seconds with numbers you can hold in your head, while CAT wants a question where the obvious approach is the slow one and there is an insight that collapses the work. Banking rewards approximation over calculation. Placement papers stick to one concept per question. Choosing the wrong one gives you a technically correct question aimed at nobody.

What the tool is told to do differently. Aptitude questions are marked against a clock, so the question has to be solvable in the time that paper allows — and the wrong options are treated as part of the lesson, not padding. Each one has to be a mistake candidates genuinely make: the ratio inverted, the percentage taken on the wrong base, the units left unconverted, the off-by-one in a series, the answer to the question that was not asked. A

distractor nobody would pick makes the question easier than the real paper and teaches nothing.

The explanation is told to name the method out loud — "this is an alligation", "use the LCM method" — because the pattern is the takeaway, not this one answer. Where a shortcut exists it shows the textbook route first, then the shortcut, and says how much time it saves.

Long-form method videos

This is where aptitude pays off most. Switch **How it is told** to **Explainer** and the storyboard is built as a method lesson rather than a curiosity piece:

1. Open on a question of this type and make it clear why the obvious approach is too slow.
2. Name the method in one sentence.
3. Work a full example, one step per scene, on a **steps** panel so the working builds up on screen.
4. Put the shortcut against the long way on a **versus** panel, with the seconds saved.
5. Show the trap this chapter is famous for on a **grid** panel.
6. Work a second, slightly different example so the viewer sees the pattern transfer.
7. Recap the method as numbered steps they can screenshot.

An ordinary explainer is told to use pictures and analogies instead of equations. That instruction is deliberately reversed here: in a method video the arithmetic on screen *is* the picture, and a viewer who cannot see the working cannot copy it. Keep the numbers clean enough to follow without pausing — if the working needs a calculator, the question needs different numbers.

A note on units

Voice models read **10 MW** as "ten mili wag". Any unit symbol with a number in front of it — MW, kV, kWh, MVA, Hz, Ω, °C, % and the rest — is expanded for the **voice only**. The narrator says "ten megawatts"; the screen still shows `10 MW`, and the highlight lands on it at the right moment.

A symbol with no number before it is left alone, so "A transformer" stays an article and does not become "ampere transformer". Write units the way you normally would.

---

5. Git — your undo button

Git takes **snapshots** of your project. It is the reason a mistake can never cost you more than a few minutes.

Three ideas, and that is genuinely all you need:

- **A commit** is a labelled snapshot. "Here is what everything looked like at 4pm."
- **Your history** is the list of those snapshots.
- **You can always go back** to any of them.

Git deliberately ignores three things: `node_modules` (hundreds of megabytes, rebuilt by `npm install`), your finished videos, and generated audio. Everything that matters is tracked.

The only commands you need

See what you have changed:

```
git status
```

Stage everything you changed:

```
git add -A
```

Save the snapshot:

```
git commit -m "describe what you changed"
```

The message is for future-you. "Made the countdown longer" beats "update".

See your history:

```
git log --oneline
```

Undoing things

Throw away changes to one file you have messed up:

```
git restore path/to/file
```

Throw away **all** uncommitted changes and go back to your last snapshot:

```
git restore .
```

> ⚠ That last one is not itself undoable. It discards everything you have changed since your last commit. Commit often and it is never frightening.

---

6. GitHub — your backup and your bridge

GitHub is a website that stores a copy of your project in the cloud. It does two jobs: it is a backup if your laptop dies, and it is how the same project reaches a second computer.

Your project lives at **github.com/HHHkumar/shorts-studio**, and it is **private** — only you can see it.

Send your latest snapshots up:

```
git push
```

Bring down changes made on another machine:

```
git pull
```

The rhythm that keeps it painless

Pull before you start. Push before you stop.

A full session looks like this:

```
git pull
```

...do your work, then...

```
git add -A
```

```
git commit -m "what you changed"
```

```
git push
```

If you forget and edit the same file on both machines, Git will ask you to reconcile the two versions. Irritating, but nothing is ever lost.

> **If a push fails with "Permission denied to <some other name>"**, Windows has a different GitHub account saved. That is exactly why your project's address includes your username: `https://HHHkumar@github.com/HHHkumar/shorts-studio.git`. It tells Git which account to use.

---

7. Running it on another computer

Everything you need is on GitHub, so this is four commands.

1. Open PowerShell where you want the project to live, then:

```
git clone https://HHHkumar@github.com/HHHkumar/shorts-studio.git
```

2. Go into it:

```
cd shorts-studio
```

3. Install the packages (this is why `node_modules` is not in the repo — it is rebuilt here):

```
npm install
```

4. Start it:

```
npm start
```

Four things that do not travel, by design

- **Your API keys.** They live in your browser, never in a file. Paste them into step 1 again.
- **The rendering browser.** The first render on the new machine downloads it again (~150 MB, once).

- **Your finished videos.** They are too big for a repo. They stay on the machine that made them.
- **The Python packages for the guide PDF.** `npm install` fetches everything JavaScript, but the one Python script in the project — the one that rebuilds this document as a PDF — has its own two dependencies. You only need them if you edit the guide:

```
pip install -r tools/requirements.txt
```

After that, both computers are equal. Pull before you start, push before you stop, and they stay in step.

---

8. What it costs

Service	Free allowance	One video uses
Gemini	A generous free tier	One or two requests. Realistically free.
ElevenLabs	10,000 characters a month	A short uses 500–900 characters (**10–20 a month free**). A five-minute explainer uses about 4,400 (**two a month**).
Claude *(optional)*	Pay as you go, **no free tier**	Opus: about **9c** a quiz, **35c** a five-minute storyboard. Sonnet: about **4c** and **14c**.
DeepSeek *(optional)*	Pay as you go, no free tier	A fraction of a cent per check.
Pexels / NASA *(optional)*	Free	Nothing.
Drawn backdrops *(optional)*	**None — needs an ElevenLabs Pro plan**	One press, one picture. Spends the same credits as the voice, so draw only the scenes that need it.
Rendering	Unlimited	Your own computer. Costs electricity.

Changing the look, re-rendering, editing text and picking photos all cost **nothing**, and neither does **Write it myself** — that path needs no Gemini key at all. Only *Generate the question*, *Make the voiceover*, *Check the answer* and *Publish* spend anything.

To stretch ElevenLabs credits: shorter targets, and the **Flash** voice model.

---

9. When something goes wrong

What you see	What it means	What to do
"*running scripts is disabled on this system*"	Windows blocks npm's launcher by default.	Use `npm.cmd`, or run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once.
Red box: "*The helper server is not running*"	The helper really is not there — the page waited twenty seconds first.	Reopen PowerShell, `cd` to the folder, `npm start`, reload the page.
"*Port 3030 is already taken*"	A second copy of the tool is still running.	Close the other PowerShell window. The message prints the command to force it if you cannot find it.

What you see	What it means	What to do
"*Port 5173 is already in use*"	Same thing, for the web half.	Close the other window and start again — but see the next row if that does not help.
The port is still taken after you closed the window	Closing a window does not always kill what it started. `npm start` runs **two** child processes, and they can outlive the window and keep listening.	Find and stop them by number — see When a port stays stuck below.
Both halves exit at once, `[APP]` then `[SERVER]`	Normal, and not two faults. The two halves run under `concurrently -k`, so when one fails it deliberately stops the other.	Read the **first** error only. The second exit is the consequence, not the cause.
"*That Claude API key was rejected*"	Bad key, usually a stray space.	Re-copy it from console.anthropic.com.
"*Your Anthropic account has no credit left*"	Claude has no free tier.	Top up, or switch **Who writes it** back to Gemini on step 2.
"*Claude declined this topic*"	A safety classifier refused it.	Reword the topic, or use Gemini for that one.
The **Who writes it** choice is missing	No Claude key is set.	Paste one on step 1; the choice appears by itself.
"*This model spent its whole budget thinking*"	A newer Flash model reasoned until it had no room left to write.	Pick a **2.5** model in the dropdown. They are the reliable choice for long scripts.
"*No reply after 240 seconds*"	The model stalled.	Try again, or switch to a Flash model.
"*That … API key was rejected*"	Bad key, usually a stray space.	Re-copy it from the provider and paste again.
"*Your ElevenLabs character quota is used up*"	Out of voice credits this month.	Wait for the reset, shorten the video, or upgrade.
"*ElevenLabs image generation needs a Pro plan or above*"	Drawing backdrops is a paid-plan feature on the API. Your voiceover is unaffected.	Use **Find backdrop photos** instead, or upgrade the plan.
"*ElevenLabs refused that prompt as unsafe*"	A moderation filter rejected the words for that scene.	Reword that scene's search box and press **Draw** again.
"*ElevenLabs was still drawing after 120 seconds*"	The model queued rather than failed.	Press **Draw** again, or pick a **Flash** image model — they answer in seconds.
The **Draw** button is missing on step 5	No ElevenLabs key is set.	Paste one on step 1; the button and its two dropdowns appear by themselves.
`public\generated\ai\` keeps growing	Drawn images are never deleted automatically, because they cost credits.	Delete the folder yourself when you have finished with those videos.
"*Your DeepSeek account has no credit left*"	DeepSeek has no free tier.	Top up, or clear the key to turn the check off.
"*Gemini rate limit hit*"	You generated too fast.	Wait a minute and try again.

What you see	What it means	What to do
"*Your key cannot use that Gemini model*"	The model is not available to your key.	Reload the page — the dropdown rebuilds from your own key.
No voices in the dropdown	Your ElevenLabs account has no voices saved.	Add one from the Voice Library, reload.
Render stuck at 0% the first time	It is downloading the rendering browser.	Wait. It happens once per computer.
"*The output file is locked*"	A video player still has the last render open.	Close it and render again.
A spoken line sounds clipped	Trailing silence trimmed too aggressively.	Step 5 → turn off **Trim trailing silence**.
A diagram shows wrong numbers	Gemini invented them.	Fix that scene in step 3, or turn off **Draw the diagrams**.
A video feels too busy	Too much is moving for the subject.	Step 5 → **Narration effects** → **How strong**. Turn it down; 0 holds everything still.
A scene feels flat and still	The narration names no movement.	Step 3 → check the ✨ **Moves on these words** row. Use the real verb: *flows*, not *goes*.
An effect fires where it makes no sense	A word matched that you did not mean physically.	Reword that phrase; the row shows which word did it.
A moving scene is out of step with the voice	A cue is missing from that scene's narration.	The window says which words. Put them back, or regenerate.
A box has no picture in it	No icon matched that noun, or the storyboard named an abstract idea.	Normal for abstractions. Otherwise use a plainer noun.
A moving scene has a blank circle in it	No icon matched that word.	Step 3 → reword it to a plainer noun, e.g. "fish" not "salmonid".
No moving scenes ever appear	The subject may not have a moment that moves — or the model skipped it.	Step 2 → **Extra instructions** → *act out the mechanism with a moving scene*.
The script is far shorter than asked	Gemini underwrote it.	Step 3 warns you. Regenerate, or switch to a stronger model.
Push fails: "*Permission denied to …*"	Windows has another GitHub account saved.	Make sure the address includes `HHHkumar@`.
The kit has no chapters	The video is too short, or YouTube's rules cannot be met.	Normal on Shorts. Chapters need 60s+, three marks, ten seconds each.
Pack the upload kit is greyed out	The metadata has not been written yet.	Press **Write the metadata** first — the kit is built from it.
The kit has no thumbnail in it	It was packed before you made one.	Make the thumbnail, then press **Pack it again**.
A change seems to have no effect	An old server is still running from before.	`Ctrl + C` in PowerShell, then `npm start` again.

What you see	What it means	What to do
Everything is confusing	—	Step 7 → **Reset everything**, then start from step 1.

When a port stays stuck

`npm start` is not one program. It starts **two** — the helper server on 3030 and the web app on 5173 — and closing the window they were launched from does not always take them with it. They keep running, invisibly, still holding both ports. The next `npm start` then fails on 5173 before it has done anything, and because the two halves are deliberately tied together, the helper is stopped too. That is why you get two red exits for one problem.

Find out what is actually holding the ports:

```
netstat -ano | findstr "5173 3030"
```

The last number on each line is the process ID. Stop each one:

```
taskkill /PID 21044 /F
```

Then `npm start` as normal. Nothing is lost by doing this — those processes hold no unsaved work. Your videos are in `out\`, your keys are in the browser, and everything else is on disk already.

> If this happens every time you close the window, get into the habit of stopping the tool with `Ctrl + C` in its own window rather than clicking the X. `Ctrl + C` shuts both halves down properly; the X sometimes only closes the window around them.

Checking the build itself

If something feels broken and you want to know whether it is the tool or the model, run:

```
npm test
```

The fastest one. It checks the rules — the cleaners, the timings, the chapter arithmetic, the animation pacing — without drawing anything. A few seconds, no API key, no network.

```
npm run simulate
```

Feeds a complete storyboard — deliberately including a few of the mistakes a model really makes — through every stage the real thing uses: the cleaner, the motion check, the icon fetch, the timeline, the upload kit and the renderer. Add `-- --render` to also encode a real video from it, which takes a few minutes and proves the whole chain end to end.

```
npm run smoke
```

The widest one, and the one to run after any change to how videos are put together. It renders **both kinds of video in both shapes** — quiz and explainer, 9:16 and 16:9 — using an explainer that deliberately contains every

layout, plus all eight thumbnails. It then checks that no frame came out blank, which is the failure that hides: a scene that renders successfully and draws nothing looks exactly like a scene that is fine, until you watch it.

It leaves the frames in `stills\smoke\` so you can look at them yourself. That is worth doing — the checks catch a blank frame, they cannot tell you a layout is ugly.

Every line of all three should say `ok`.

---

10. Where things are saved

Inside `C:\Projects\shorts-studio`:

- `out\` — your finished videos. **This is the folder you want.** Nothing here is ever deleted automatically, and nothing here goes to GitHub.
- `public\generated\` — voiceover clips and backdrop images.
  - `vo-…\` and `stock\` are **cleared automatically after a day**. Both are free to make again.
  - `ai\` is **never cleared**, because those images cost ElevenLabs credits and deleting them on a timer would spend your money twice. It grows until you empty it yourself.
- `public\audio\` — the music beds and effects, regenerated on first boot.
- `.cache\` — the icon drawings, kept so the same noun is never fetched twice. Safe to delete; it refills itself.
- `stills\` — frames written by the checks in section 9. Scratch, safe to delete.
- `.git\` — your snapshots. Do not touch it; that is Git's business.
- Your keys and settings — in your browser, not in any file.

---

11. Questions people ask

Can I use my own voice? Yes. Clone it on the ElevenLabs website and it appears in the step 4 dropdown.

Can I change the fonts and colours? Yes. `src\lib\theme.ts` holds every visual choice, with comments. Save it and the preview updates instantly. If you break it, `git restore src/lib/theme.ts` puts it back.

Does it work offline? No. Gemini, ElevenLabs and the one-time renderer download all need the internet. Re-rendering a video you already generated works offline.

Is my data going anywhere? Your topic settings go to Google, your script to ElevenLabs, your question to DeepSeek if you enabled it, and — only if you press **Draw** — that scene's few words go to ElevenLabs as an image prompt. Nothing else leaves the computer — the helper server listens only on `127.0.0.1`, so nothing on your network can reach it.

Why is my project not in OneDrive any more? OneDrive was syncing `node_modules` — tens of thousands of files — which is slow and can lock files mid-render. Git does the job properly, so the project moved to `C:\Projects`.

Can I sell the videos I make? Check each service's terms. Note in particular that **Remotion is free for individuals and small teams but needs a paid company licence beyond that** — see https://remotion.dev/license.

Something is still wrong. Look at the PowerShell window. The last few red lines usually say plainly what failed.

---

12. A checklist for good videos

- Curiosity factor at **8 or 9**. Boring questions do not get watched.
- 40–50 seconds for Shorts; 180–240 for explainers.
- **Read the question in step 3 and verify the answer yourself.**
- Run the DeepSeek check — a disagreement between two models is the cheapest bug report you will get.
- Keep the read-along text on; most viewers are on mute.
- Only pick backdrop photos that genuinely fit. An unrelated one makes it look worse, not better.
- If you draw backdrops, keep one **Look** for the whole video. Mixed styles read as carelessness.
- For an aptitude video, check the wrong options are real mistakes. If one is obviously silly, the question is easier than the paper it is meant to prepare you for.
- Thinking time of 3–5 seconds. Longer and people scroll away.
- **Commit and push when you finish.** It takes ten seconds and it is the whole safety net.